Relatório

TPC 3

Pedro Manuel Fernandes Magalhães | 202405428

Log das atividades e metodologia

1. Definição de objetivos e planeamento

O primeiro passo consistiu em analisar a aplicação e determinar que tipo de teste de aceitação seria mais valioso para uma interface minimalista. Sendo a app focada na exibição de uma mensagem estática, o objetivo central foi definido como a verificação da fidelidade visual e estrutural. Desse modo, decidi utilizar o framework oficial do Flutter, o integration_test, devido à sua integração nativa e estabilidade, dispensando ferramentas externas mais pesadas como o Selenium ou Appium.

2. Configuração do ambiente e ferramentas

Para preparar o projeto para a automação, foi necessário ajustar o ficheiro de configuração de dependências. Adicionei o pacote de testes de integração nas dev_dependencies do projeto, conforme documentado na documentação oficial do Flutter sobre testes de integração/aceitação.

Instruções:

(1) Abrir o ficheiro pubspec.yaml;

(2) Encontrar a secção dev_dependencies;

(3) Adicionar:
integration_test:
sdk: flutter

Exemplo:

dev_dependencies:

flutter_test:

sdk: flutter

-> integration_test:

-> sdk: flutter

Comandos para correr os testes

(1) **flutter clean** (opcional, usar quando existirem problemas na build)

(2) **flutter pub get** (se o flutter clean for usado, deve ser usado este comando para repor as dependências)

(3) **flutter test integration_test/app_test.dart**

Material de apoio: Consultei a documentação da Google para entender o funcionamento do WidgetTester e do IntegrationTestWidgetsFlutterBinding, Essa consulta à documentação técnica foi um passo determinante para garantir o rigor metodológico da automação, sem esquecer que a compreensão do IntegrationTestWidgetsFlutterBinding permitiu configurar corretamente o ambiente de execução em hardware real/emulado, assegurando que os testes de aceitação não fossem meras simulações lógicas, mas sim validações do comportamento da interface em tempo de execução. Complementarmente, o domínio do WidgetTester permitiu o controlo programático do ciclo de vida dos widgets, possibilitando a validação de estados e propriedades visuais de forma síncrona e fiável.

3. Especificação do cenário de aceitação

Utilizando a linguagem Gherkin como referência para manter a objetividade, especifiquei o cenário de sucesso. O foco foi garantir que a AppBar continha o título correto ("Hello") e que a mensagem central não só existia, mas possuía os atributos de estilo definidos no código original (tamanho de fonte 28 e peso w500), mas para além disso, durante a implementação, também se identificou a necessidade de expandir os testes para validar também as propriedades de cor, mais especificamente a cor laranja configurada no tema. Num cenário real de desenvolvimento ágil, alterações acidentais nas cores de branding por parte da equipa de design ou desenvolvimento são comuns e muitas vezes passam despercebidas em testes unitários, assim ao garantir a cor do tema através de um teste de aceitação automatizado, asseguramos que a identidade visual da aplicação (Branding) se mantém intacta e que o contraste esperado para a leitura do texto sobre o fundo branco/claro do Scaffold é preservado.

Esta fase de especificação é crucial para transformar um requisito subjetivo ("a app deve estar bonita") num critério de aceitação técnico e mensurável.

4. Implementação da automação

A codificação dos testes foi realizada no diretório tpc_3/integration_test/app_test.dart, neste seguimento, durante a escrita do código, utilizei técnicas de inspeção de widgets para validar propriedades internas que não são visíveis apenas com um simples "find text". Para além disso com o objetivo de auxiliar na verificação de qualidade dos testes por mim redigidos, recorri a ferramentas de suporte à programação (GenAI), o que me permitiu validar, até certo ponto, a exatidão dos meus testes tendo em conta o contexto onde estavam inseridos.

5. Controlo de versões e submissão (GitHub)

Para cumprir os requisitos de entrega e garantir o versionamento correto do código de testes, todo o trabalho foi registado e submetido para o repositório do projeto no GitHub.

Comandos executados no terminal:

Criação e mudança para uma nova branch pessoal: git checkout -b <nome_da_branch>

Adição de novos ficheiros à staging area: git add . ou git add <filename>

Registo dos commits com uma mensagem descritiva e clara: git commit -m "<mensagem>"

Envio (push) da branch para o repositório remoto: git push origin <nome_da_branch>

Para aceder ao meu repositório basta clicar aqui.

Análise Crítica do Processo

A transição de uma especificação manual para uma automação funcional revelou-se um exercício de rigor técnico. Inicialmente, poderia até parecer redundante testar uma aplicação com tão pouca interatividade, no entanto, a minha análise crítica do processo demonstra que a automação de testes de aceitação é a forma mais viável de garantir a manutenibilidade de um projeto a longo prazo.

Um dos principais desafios encontrados foi a necessidade de referenciar os widgets de forma inequívoca, e no desenvolvimento inicial, não foram utilizadas Keys, o que dificultou ligeiramente a identificação de elementos específicos. Este obstáculo serviu como uma lição aprendida: para facilitar a testabilidade (Design for Testability), sendo recomendável atribuir identificadores únicos aos widgets, mesmo em apps mais simples.

A escolha do integration_test em detrimento do Patrol ou Selenium revelou-se acertada para este contexto específico, pois permitiu uma execução rápida e sem a complexidade de configurar drivers externos. Em suma, o processo provou que a qualidade de software não depende da complexidade da aplicação, mas sim da robustez dos mecanismos que garantem que os requisitos, por mais minimalistas que sejam, são rigorosamente respeitados em cada ciclo de desenvolvimento.

Sites usados:

- https://docs.flutter.dev/testing/integration-tests
- https://docs.flutter.dev/testing
- https://docs.flutter.dev/get-started/learn-flutter
- https://docs.flutter.dev/get-started/codelab
- https://docs.flutter.dev/reference/learning-resources

Prompts usados na GenAI (ChatGPT):

Inputs:

- "Analisa estes testes de aceitação de Flutter e diz se está tudo bem estruturado e fácil de manter"
- "Verifica se estes testes de aceitação cobrem os cenários mais importantes da app, se não, sugere casos de teste adicionais que poderiam ser incluídos para melhorar a cobertura funcional."
- "Sugere melhorias de legibilidade e organização para este testes de aceitação Flutter"

Outputs:

Forneceu apenas algumas pequenas melhorias opcionais a nível organizacional.